SINGLE RESOURCE WITH MULTIPLE URLs IN ASP DOT NET CORE WEB API

In RESTful API design, different clients often expect different endpoints names for the same resource. For example:

- Admin dashboard wants */api/books/GetAll*
- Mobile app wants */api/books/AllBooks*
- Legacy system wants */api/books/All*

Instead of creating three separate methods with duplicate code, ASP.NET Core lets us decorate **one method** with **route attributes** – all pointing to the same logic.

We will perform basic tasks like create and navigate to the project, add packages and build etc. using the cli/Windows powerShell.

Now we are going to set up the project,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n LibraryApi2 -f net8.0

<u>Navigate to the project:</u>

Command: cd LibraryApi2

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'LibraryApi2' in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

LibraryApi2.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Define the Model

Path: LibraryApi2/Models/Book.cs

```
Book.cs
Models > Book.cs > ...
namespace LibraryApi2.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

What is this?

The data blueprint for our API. Defines what a “Book” looks like – each property becomes a field in our API response.

Step 2: Initialize In-Memory Data

Path: LibraryApi2/Data/BookData.cs

```
BookData.cs ×
Data > BookData.cs > ...
using LibraryApi2.Models;

namespace LibraryApi2.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new List<Book>
        {
            new Book {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
            new Book {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
            new Book {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
            new Book {
                Id = 6,
                Title = "Moby Dick",
                Author = "Herman Melville",
                Genre = "Adventure",
                PublishedYear = 1851,
                IsAvailable = false,
                Price = 14.99m
            },
            new Book {
                Id = 7,
                Title = "The Shining",
                Author = "Stephen King",
                Genre = "Horror",
                PublishedYear = 1977,
                IsAvailable = true,
                Price = 13.99m
            }
        };
    }
}
```

What is this?

A temporary, in-memory database. Instead of connecting to SQL Server, we store data in a static list for testing. Perfect for learning and prototyping.

Step 3: Program.cs (Minimal Setup)

Path: LibraryApi2/Program.cs

```
Program.cs
Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

What is this?

The application startup file. It registers controllers, enables Swagger for testing, and sets up the HTTP request pipeline.

Step 4: Implement Multiple Routes

Path: LibraryApi2/Controllers/BooksController.cs

BooksController.cs

Controllers > BooksController.cs > ...

```
1   using LibraryApi2.Data;
2   using LibraryApi2.Models;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace LibraryApi2.Controllers
6   {
7       [ApiController]
8       [Route("api/[controller]")]
        0 references
9       public class BooksController : ControllerBase
10      {
11          // THREE URLs point to this SINGLE method
12          [HttpGet("All")]            // URL  1: /api/books/All
13          [HttpGet("AllBooks")]       // URL  2: /api/books/AllBooks
14          [HttpGet("GetAllBooks")]    // URL  3: /api/books/GetAllBooks
            0 references
15          public ActionResult<IEnumerable<Book>> GetAllBooks()
16          {
17              var books = BookData.Books;
18              return Ok(books);
19          }
20      }
21  }
22
```

What is happening here?

- One method named *GetAllBooks*
- Three *[HttpGet]* attributes with different route strings
- All three URLs execute the EXACT SAME code
- Returns all 7 books from our in-memory data

What if Two Methods Use the Same Route?

This causes an error.

```
// WRONG - Both methods have route "All"
[HttpGet("All")]
public ActionResult<IEnumerable<Book>> GetAllBooks() { ... }

[HttpGet("All")]  // SAME ROUTE!
public ActionResult<IEnumerable<Book>> GetAvailableBooks() { ... }
```

Result: ***AmbigousMatchException*** – ASP.NET Core doesn't know which method to call.

FIX: Each route string must be unique across all action methods.

Backward Compatibility Scenario

When renaming an endpoint, keep old one for existing clients.

```
[ApiController]
0 references
public class BooksController : ControllerBase
{
    [Route("api/old-books-list")]      // Old clients still work
    [Route("api/library/books")]       // New preferred name
    [Route("api/books/getall")]        // Alternative naming
    [HttpGet]
    0 references
    public ActionResult<IEnumerable<Book>> GetAllBooks()
    {
        return Ok(BookData.Books);
    }
}
```

Why?

If we change, */api/old-books-list* to, */api/library/books*, existing mobile apps would break. Keeping both allows gradual migration.

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi1) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi2> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi2 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi2\bin\Debug\net8.0\Lib
  raryApi2.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:03.83
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi2>
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi2> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/do
tnet/hot-reload.
  💡 Press "Ctrl + R" to restart.
dotnet watch 🔧 Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi2 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi2\bin\Debug\net8.0\Lib
  raryApi2.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5094
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi2
```

TEST 1: First URL - /api/books/All

Request: GET **http://localhost:5094/api/Books/All**

Response:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5094/api/Books/All' \
  -H 'accept: text/plain'
```

Request URL

```
http://localhost:5094/api/Books/All
```

Server response

Code	Details
200	

Response body

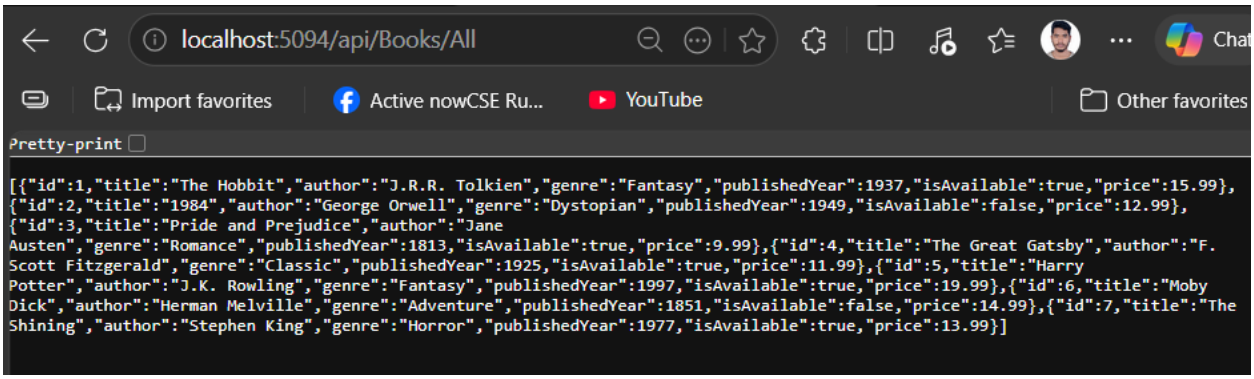

```
[
  {
    "id": 1,
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "genre": "Fantasy",
    "publishedYear": 1937,
    "isAvailable": true,
    "price": 15.99
  },
  {
    "id": 2,
    "title": "1984",
    "author": "George Orwell",
    "genre": "Dystopian",
    "publishedYear": 1949,
    "isAvailable": false,
    "price": 12.99
  },
  {
    "id": 3,
    "title": "Pride and Prejudice",
    "author": "Jane Austen",
    "genre": "Romance",
    "publishedYear": 1813,
    "isAvailable": true,
    "price": 9.99
  },
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue,14 Apr 2026 08:58:57 GMT
server: Kestrel
transfer-encoding: chunked
```

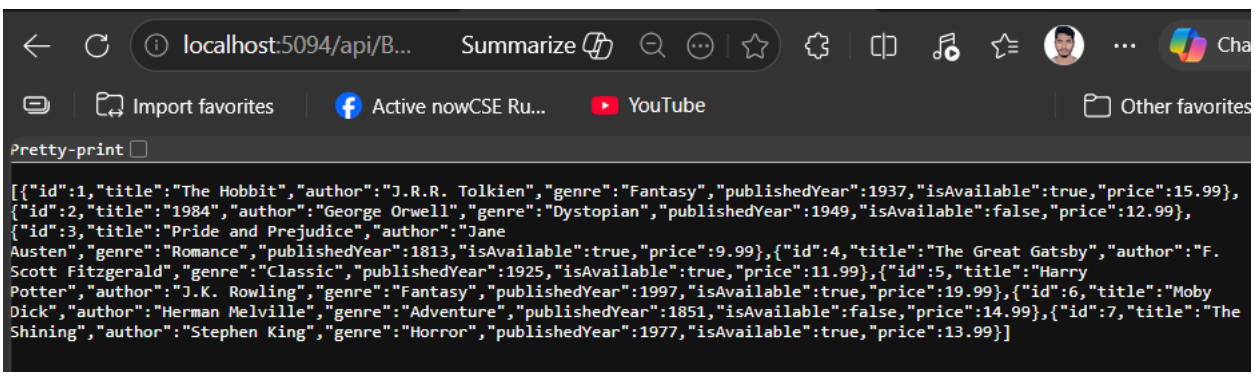
Full Response:



TEST 2: Second URL - /api/books/AllBooks

Request: GET **http://localhost:5094/api/Books/AllBooks**

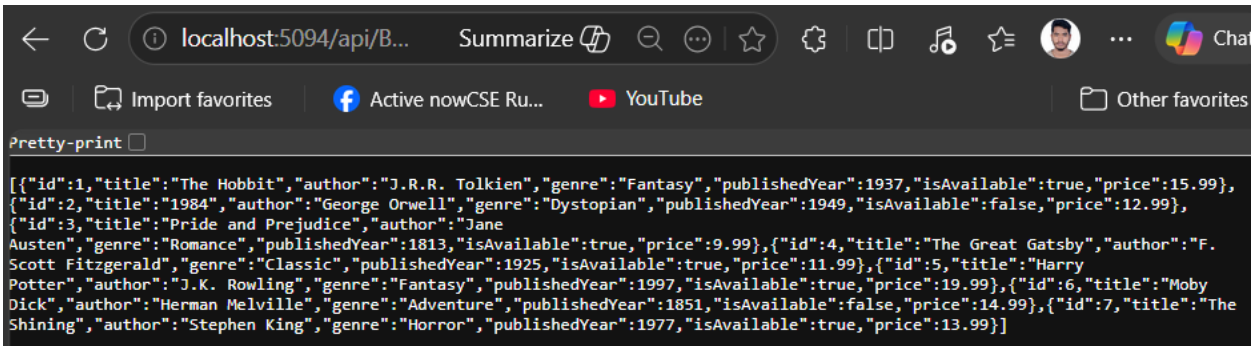
Full Response:



TEST 3: Third URL - /api/books/GetAllBooks

Request: GET **http://localhost:5094/api/Books/GetAllBooks**

Full Response:



Close the application if everything works successfully.

Key Takeaways

Concept	Explanation
Multiple URLs per Resource	Decorate one method with multiple *[HttpGet]* attributes
Must Use Attribute Routing	Convention routing doesn't support this
Avoid Duplicate Routes	Two methods cannot share the same route string
Backward Compatibility	Keep old + new routes during API evolution
No Code Duplication	Same logic serves all URLs